1

Introdução aos paradigmas de programação

Universidade Estadual da Paraíba (UEPB)
Paradigmas de Programação
Kláudio Henrique Mascarenhas Medeiros

Paradigmas de Programação (discip.)

- Linguagens de programação... é sobre isso (e tá tudo bem).

Paradigmas de Programação (discip.)

- Linguagens de programação... é sobre isso (e tá tudo bem).

- As aulas abordarão diversas características de linguagens de programação das mais diversas.

Paradigmas de Programação (discip.)

- Linguagens de programação... é sobre isso (e tá tudo bem).

- As aulas abordarão diversas características de linguagens de programação das mais diversas.

- Além disso, vamos fugir um pouco do que estamos acostumados, estudando, por ex., linguagens funcionais, lógicas e orientadas a eventos.

Paradigmas de Programação (discip.)

- O que faremos?
    - Unidade I: documento escrito sobre uma linguagem de programação e um jogo desenvolvido nela

Paradigmas de Programação (discip.)

- O que faremos?
  - Unidade I: documento escrito sobre uma linguagem de programação e um jogo desenvolvido nela
  - Unidade II: documento escrito sobre aspectos de linguagens funcionais e lógicas

Paradigmas de Programação (discip.)

- Algumas linguagens são proibidas. São essas: Python, C, Java, Javascript, C++, C#, Rust, Lua, PHP, Pascal, Ruby, Swift, Delphi, Dart, Typescript, Crystal, Go, Kotlin e Scala.

Paradigmas de Programação (discip.)

- Algumas linguagens são proibidas. São essas: Python, C, Java, Javascript, C++, C#, Rust, Lua, PHP, Pascal, Ruby, Swift, Delphi, Dart, Typescript, Crystal, Go, Kotlin e Scala.

- O jogo que cada grupo irá desenvolver é o Uno.

Paradigmas de Programação (discip.)

- Algumas linguagens são proibidas. São essas: Python, C, Java, Javascript, C++, C#, Rust, Lua, PHP, Pascal, Ruby, Swift, Delphi, Dart, Typescript, Crystal, Go, Kotlin e Scala.

- O jogo que cada grupo irá desenvolver é o Uno.

- Sobre as atividades da Unidade II, mais detalhes sobre elas serão dados mais adiante.

Linguagens de programação

- Você, enquanto programador, irá recorrer ao computador para resolver um problema.

Linguagens de programação

- Você, enquanto programador, irá recorrer ao computador para resolver um problema.

- Você pensa de uma maneira que o computador dificilmente vai entender.

Linguagens de programação

- Você, enquanto programador, irá recorrer ao computador para resolver um problema.

- Você pensa de uma maneira que o computador dificilmente vai entender.

- Você precisa se comunicar com o computador de alguma forma, e utiliza para isso uma linguagem, que é a **linguagem de programação**.

Paradigmas de programação

- O paradigma de programação representa um **modelo para estruturar e representar um problema**, a ser solucionado através de um programa.

Paradigmas de programação

- O paradigma de programação representa um **modelo para estruturar e representar um problema**, a ser solucionado através de um programa.

- Existem **vários paradigmas de programação**, como imperativo, funcional, lógico e orientado a objetos.

Paradigmas de programação

- O paradigma de programação representa um **modelo para estruturar e representar um problema**, a ser solucionado através de um programa.

- Existem **vários paradigmas de programação**, como imperativo, funcional, lógico e orientado a objetos.

- Uma linguagem de programação pode **dar suporte a um, bem como a vários paradigmas**, fazendo dela uma linguagem **multiparadigma** (e.g. Python).

Paradigmas de programação

Linguagens imperativas utilizam comandos, linguagens funcionais usam funções e expressões, linguagens lógicas utilizam fatos e regras, etc.

Por que se preocupar em estudar isso?

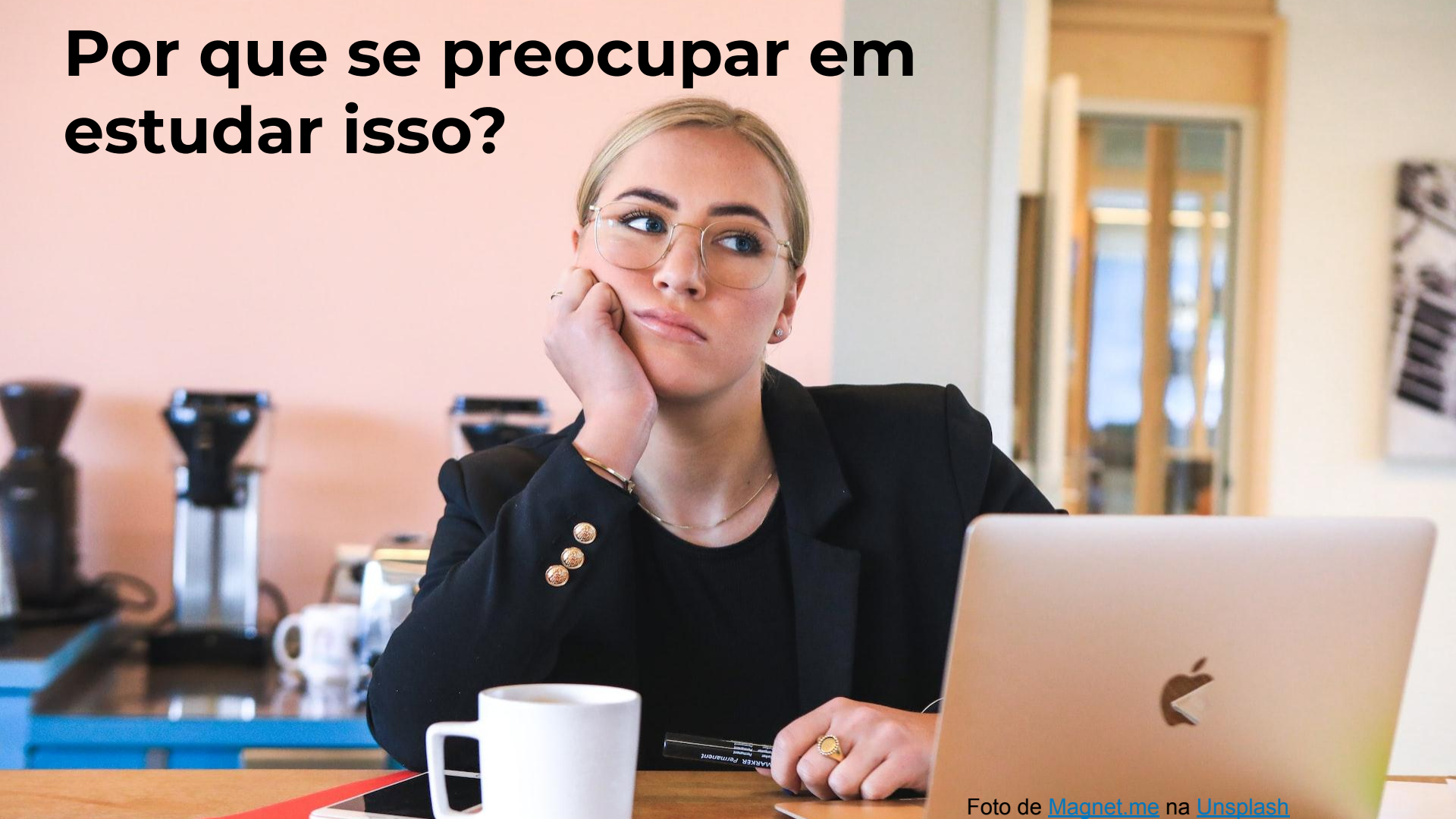


Foto de Magnet.me na Unsplash

Por que estudar paradigmas de prog.?

- Nossa capacidade intelectual aumenta conforme aprendemos novas linguagens (e.g. idiomas, de prog.)

Por que estudar paradigmas de prog.?

- Nossa **capacidade intelectual** aumenta conforme aprendemos novas linguagens (e.g. idiomas, de prog.)

- Encontrar **diferentes formas de resolver um problema e escolher a mais adequada delas** (e.g. como representar um Node? - ponteiro? lista? classe?)

Por que estudar paradigmas de prog.?

```
struct node {
   int value;
   struct node *nextPtr;
};
```

construção de um Node em C
- quais as características dessa construção?

```
public class Node {
   private int valor;
   private Node next;
   ...
}
```

construção de um Node em Java
- quais as características dessa construção?

Por que estudar paradigmas de prog.?

- Nossa **capacidade intelectual** aumenta conforme aprendemos novas linguagens (e.g. idiomas, de prog.)

- Encontrar **diferentes formas de resolver um problema e escolher a mais adequada delas** (e.g. como representar um Node? - ponteiro? lista? classe?)

- Quanto mais linguagens sabemos, **mais fácil e melhor será aprender novas linguagens**.

O quão difícil foi sair de nada para Python? E de Python para C?

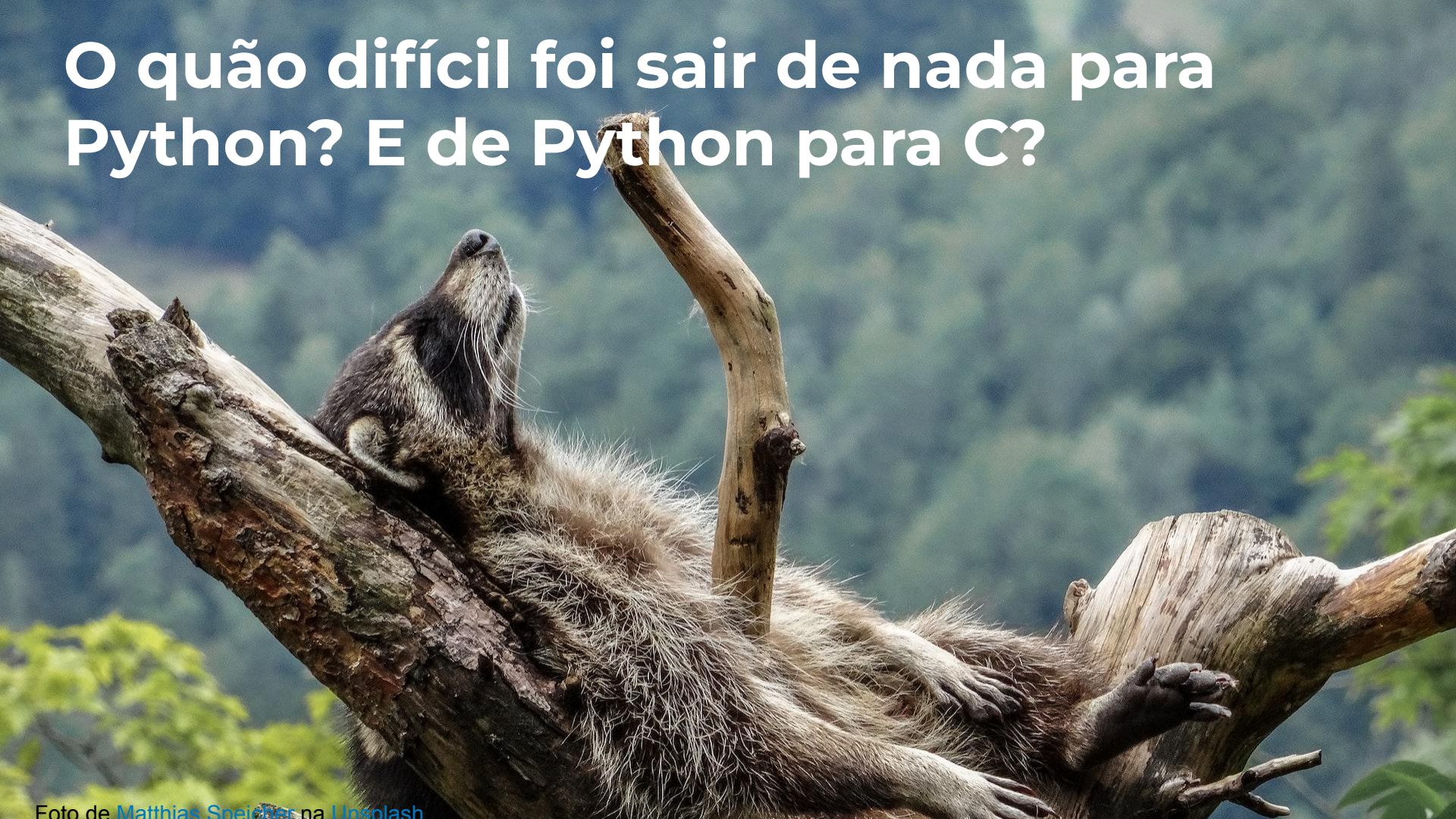


Foto de Matthias Speicher na Unsplash

Um paralelo com os idiomas...

- “Eu sou Kláudio” em português. Segue uma estrutura sujeito (eu), verbo (sou) e objeto (Kláudio).

Um paralelo com os idiomas...

- “Eu sou Kláudio” em português. Segue uma estrutura sujeito (eu), verbo (sou) e objeto (Kláudio).

- “I am Kláudio” em inglês. Segue a mesma estrutura. O mesmo para o francês (“Je suis Kláudio”) ou espanhol (“Yo soy Kláudio”).

Um paralelo com os idiomas...

- “Eu sou Kláudio” em português. Segue uma estrutura sujeito (eu), verbo (sou) e objeto (Kláudio).

- “I am Kláudio” em inglês. Segue a mesma estrutura. O mesmo para o francês (“Je suis Kláudio”) ou espanhol (“Yo soy Kláudio”).

- “私はクラウディオです” em japonês (lê-se “*watashi wa Kuraudio desu*”). Segue estrutura parecida, mas no japonês o objeto precede o verbo.

A estrutura difere, mas a ideia não

Foto de Christina @ wocintechchat.com na Unsplash

Um paralelo com os idiomas...

No português, temos a palavra “saudade”. Qual o equivalente a “saudade” no inglês?

Voltando para as LPs...

- Ah, estudar recursão é chato e difícil, né? Entendê-las abre possibilidades de programação funcional.

Voltando para as LPs...

Vocês verão que linguagens funcionais em geral nem ao menos tem iteração

Voltando para as LPs...

- Ah, estudar recursão é chato e difícil, né? Entendê-las abre possibilidades de **programação funcional**.

- É comum que um **recurso em uma linguagem seja apenas simulado em outra** (ou tenha uma forma de representar o recurso com alguma aproximação). Mas simulações costumam ser inferiores ao que simulam.

Voltando para as LPs...

C simula passagem por referência a funções utilizando ponteiros, mas ainda é uma passagem por cópia

Voltando para as LPs...

- Ah, estudar recursão é chato e difícil, né? Entendê-las abre possibilidades de **programação funcional**.

- É comum que um **recurso em uma linguagem seja apenas simulado em outra** (ou tenha uma forma de representar o recurso com alguma aproximação). Mas simulações costumam ser inferiores ao que simulam.

- LP2 é uma disciplina de OO e não de Java. Caso o aluno precise estudar C++, ele tem a base para isso.

Voltando para as LPs...

- Estudar uma nova linguagem pode deixar você **melhor em linguagens que você já conhece** (e.g. conhecer funções lambda depois de estudar o básico de Python).

Voltando para as LPs...

- Exercício: escreva um programa Python que gere uma lista que tenha os quadrados de todos os números de 1 a 10 (incluindo estes)

- A lista em questão é [1, 4, 9, 16, 25, 36, 49, 64, 81, 100]. A questão é: como seria o programa?

Voltando para as LPs...

```
# Uma lista com todos os quadrados de números entre 1 e 10
# (incluindo estes)

lista = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
lista_quad = []
for i in range(0, len(lista)): #percorrer os indexes
    lista_quad.append(lista[i] ** 2)
print(lista_quad)

lista = range(1, 11)
lista_quad = []
for elem in lista: # percorrer os elementos
    lista_quad.append(elem ** 2)
print(lista_quad)
```

Voltando para as LPs...

```
lista_quad = [elem ** 2 for elem in range(1, 11)]
print(lista_quad)

def square(x):
    return x ** 2

lista = range(1, 11)
lista_quad = list(map(square, lista))
print(lista_quad)

lista_quad = list(map(lambda x: x ** 2, range(1, 11)))
print(lista_quad)
```

Voltando para as LPs...

```
# Uma lista com todos os quadrados de números entre 1 e 10
#(incluindo estes)

square = lambda x: x ** 2
mapa = lambda funcao, lista: [funcao(elem) for elem in lista]

lista_quad = mapa(square, range(1, 11))
print(lista_quad)
```

Voltando para as LPs...

```
1
2  main :: IO()
3
4  lista_res = map (\x -> x * x) [1..10]
5
6  main = print(lista_res)
```

Terminal

```
[1 of 1] Compiling Main             ( main.hs, main.o )
Linking main ...
[1,4,9,16,25,36,49,64,81,100]
```

Exercício

Java passou a suportar funções lambda a partir do Java 8. Faça o programa anterior em Java utilizando função lambda

Voltando para as LPs...

- Estudar uma nova linguagem pode deixar você **melhor em linguagens que você já conhece** (e.g. conhecer funções lambda depois de estudar o básico de Python).

- Muito se fala de escolha de linguagens ou popularidade, mas devemos nos questionar e decidir bem por qual seguir.

Fonte: https://sustainful.tech/blog/golang-is-17-times-more-sustainable-than-python-in-depth-side-by-side-comparison/

Go is 17 times more sustainable than Python! An in-depth side by side comparison.

by sustainful | Jan 31, 2023 | Programming | 0 comments

Go é uma linguagem compilada (diferente de Python que é interpretada), com uma conversão direta para Assembly, o que acelera e muito as operações matemáticas.

Esse é um critério de escolha. Podem existir outros (e.g. popularidade, facilidade de desenvolvimento).

No que esses outros fatores poderiam influenciar na escolha da linguagem?

Domínios de programação

- Os domínios são muitos como:
  - aplicações científicas: anos 40, aritmética de ponto flutuante, estruturas mais simples (e.g. Fortran, ALGOL)

Domínios de programação

- Os domínios são muitos como:
  - **aplicações científicas**: anos 40, aritmética de ponto flutuante, estruturas mais simples (e.g. Fortran, ALGOL)
  - **aplicações empresariais**: anos 50, produção de relatórios, armazenar números e caracteres (e.g. COBOL)

Domínios de programação

- Os domínios são muitos como:
  - **aplicações científicas**: anos 40, aritmética de ponto flutuante, estruturas mais simples (e.g. Fortran, ALGOL)
  - **aplicações empresariais**: anos 50, produção de relatórios, armazenar números e caracteres (e.g. COBOL)
  - **inteligência artificial**: anos 60, construções simbólicas ao invés de numéricas (e.g. LISP, Prolog)

Domínios de programação

- Os domínios são muitos como:
  - **desenvolvimento de sistemas**: anos 60, linguagens para desenvolvimento de sistemas (e.g. C, ALGOL estendido)

Domínios de programação

- Os domínios são muitos como:
  - **desenvolvimento de sistemas**: anos 60, linguagens para desenvolvimento de sistemas (e.g. C, ALGOL estendido)
  - **software Web**: linguagens de marcação ou programação com suporte à conteúdo dinâmico (e.g. Java, JavaScript)

Alguns critérios p/ avaliação de lings.

- A **legibilidade** da linguagem é um critério (quão fácil é ler e entender o programa)
    - múltiplos recursos (e.g. operador de incremento - op++, ++op, op += 1)

Exemplo

```
int main()
{
   int a = 10;
   printf(“%d\n”, a++);
   printf(“%d\n”, ++a);
}
```

Em C, o que “a++” faz? E o que “++a” faz? Toda linguagem possui esse operador?

Essas construções têm suas utilidades, mas podem ser meio confusas.

Alguns critérios p/ avaliação de lings.

- A **legibilidade** da linguagem é um critério (quão fácil é ler e entender o programa)
    - múltiplos recursos (e.g. operador de incremento - op++, ++op, op += 1)
    - sobrecarga ou *overload* (e.g. “+” p/ int funciona diferente p/ string)

Exemplo

```
nome1 = “Ana”
nome2 = “lice”
num1 = 20
num2 = 40
print(nome1 + nome2)
print(num1 + num2)
```

Em Python, os dois usos do operador “+” são permitidos, mas produzem resultados diferentes. Podemos, inclusive, definir como o operador “+” funciona pra classes que criamos, definindo o método __add__

Alguns critérios p/ avaliação de lings.

- A **legibilidade** da linguagem é um critério (quão fácil é ler e entender o programa)

  - múltiplos recursos (e.g. operador de incremento - op++, ++op, op += 1)
  - sobrecarga ou *overload* (e.g. “+” p/ int funciona diferente p/ string)
  - evitamento de desvios (gotos) (c/ uso de funções, métodos)

Exemplo

Muito se diz que o uso de condicionais e laços dispensa o goto. Muitas linguagens nem ao menos tem um desvio assim

Alguns critérios p/ avaliação de lings.

- A **legibilidade** da linguagem é um critério (quão fácil é ler e entender o programa)

  - múltiplos recursos (e.g. operador de incremento - op++, ++op, op += 1)
  - sobrecarga ou *overload* (e.g. "+" p/ int funciona diferente p/ string)
  - evitamento de desvios (gotos) (c/ uso de funções, métodos)
  - como definir TADs e estruturas (malloc, em algumas são prontas)

Exemplo

C possui **struct**, Pascal tem **record**, Java tem **class**

Alguns critérios p/ avaliação de lings.

- A **legibilidade** da linguagem é um critério (quão fácil é ler e entender o programa)

  - múltiplos recursos (e.g. operador de incremento - op++, ++op, op += 1)
  - sobrecarga ou *overload* (e.g. "+" p/ int funciona diferente p/ string)
  - evitamento de desvios (gotos) (c/ uso de funções, métodos)
  - como definir TADs e estruturas (malloc, em algumas são prontas)
  - sintaxe (e.g. definir variáveis, blocos de código, keywords)
  - dentre outros

Exemplo

Python usa indentação para delimitar blocos de código, enquanto C utiliza chaves

Alguns critérios p/ avaliação de lings.

- A **capacidade de escrita** (*writability*) da linguagem é um critério (quão fácil é escrever o programa p/ um domínio específico)
    - poucas construções primitivas + regras para combiná-las favorece

Alguns critérios p/ avaliação de lings.

- A **capacidade de escrita** (*writability*) da linguagem é um critério (quão fácil é escrever o programa p/ um domínio específico)
  - poucas construções primitivas + regras para combiná-las favorece
  - combinações demais também pode ser um problema

Link: https://www.cadcobol.com.br/re_words.htm

Exemplo

COBOL possui mais de 400 palavras reservadas!

Alguns critérios p/ avaliação de lings.

- A **capacidade de escrita** (*writability*) da linguagem é um critério (quão fácil é escrever o programa p/ um domínio específico)
  - poucas construções primitivas + regras para combiná-las favorece
  - combinações demais também pode ser um problema
  - suporte para abstração (e.g. métodos para encapsulamento)

Exemplo

Os procedimentos (*procedures*), funções e métodos são exemplos de abstração

Alguns critérios p/ avaliação de lings.

- A **capacidade de escrita** (*writability*) da linguagem é um critério (quão fácil é escrever o programa p/ um domínio específico)
  - poucas construções primitivas + regras para combiná-las favorece
  - combinações demais também pode ser um problema
  - suporte para abstração (e.g. métodos para encapsulamento)
  - expressividade (e.g. “=” em Python p/ atrib. simples e múltipla)

Exemplo

```
a = 10
b = 15
a, b = 10, 15
```

Em Python, podemos definir o valor de a e b separadamente, bem como utilizar atribuição múltipla, de modo que 10 e 15 são atribuídos à a e b, respectivamente.

Alguns critérios p/ avaliação de lings.

- A **capacidade de escrita** (*writability*) da linguagem é um critério (quão fácil é escrever o programa p/ um domínio específico)
    - poucas construções primitivas + regras para combiná-las favorece
    - combinações demais também pode ser um problema
    - suporte para abstração (e.g. métodos para encapsulamento)
    - expressividade (e.g. "=" em Python p/ atrib. simples e múltipla)
    - confiabilidade:
        - **verificação de tipos** (tempo de compilação? execução?)

Alguns critérios p/ avaliação de lings.

- A **capacidade de escrita** (*writability*) da linguagem é um critério (quão fácil é escrever o programa p/ um domínio específico)
  - poucas construções primitivas + regras para combiná-las favorece
  - combinações demais também pode ser um problema
  - suporte para abstração (e.g. métodos para encapsulamento)
  - expressividade (e.g. "=" em Python p/ atrib. simples e múltipla)
  - confiabilidade:
    - **verificação de tipos** (tempo de compilação? execução?)
    - **manipulação de exceções** (e.g. blocos try/catch ou try/except)

Mais sobre tratamento de erros em C em https://en.wikibooks.org/wiki/C_Programming/Error_handling

Exemplo

Várias linguagens tem exceções e blocos de tratamento. C notifica erros, por ex., pelo retorno de funções (e.g. main)

Alguns critérios p/ avaliação de lings.

- A **capacidade de escrita** (*writability*) da linguagem é um critério (quão fácil é escrever o programa p/ um domínio específico)
  - poucas construções primitivas + regras para combiná-las favorece
  - combinações demais também pode ser um problema
  - suporte para abstração (e.g. métodos para encapsulamento)
  - expressividade (e.g. "=" em Python p/ atrib. simples e múltipla)
  - confiabilidade:
    - **verificação de tipos** (tempo de compilação? execução?)
    - **manipulação de exceções** (e.g. blocos try/catch ou try/except)
    - ***aliasing*** (nomes distintos referenciando mesma cél. de memória, podendo haver conflitos)

Exemplo

```
public class X {
    public int num;
    public X(int valor) { this.num = valor; } }
    public static void main(String[] args) {
        int a = 20;
        int b = a;
        a++;
        System.out.println(a + " " + b);
        X x1 = new X(10);
        X x2 = x1;
        x1.num = 19;
        System.out.println(x2.num);
    }
}
```

Em Java, b = a copia o valor de a para b. x2 = x1 copia a referência de x1 para x2. x1 e x2 apontam para o mesmo objeto, sendo, portanto, *aliases* (apelidos) para o mesmo objeto.

A saída do último println é 19.

Alguns critérios p/ avaliação de lings.

- O custo da linguagem é um critério. Esse custo pode depender de vários fatores, como:
    - o custo para treinar um desenvolvedor naquela linguagem

Alguns critérios p/ avaliação de lings.

- O custo da linguagem é um critério. Esse custo pode depender de vários fatores, como:
  - o custo para treinar um desenvolvedor naquela linguagem
  - o custo para escrever programas naquela linguagem

Alguns critérios p/ avaliação de lings.

- O custo da linguagem é um critério. Esse custo pode depender de vários fatores, como:
  - o custo para treinar um desenvolvedor naquela linguagem
  - o custo para escrever programas naquela linguagem
  - o custo para compilar e executar programas naquela linguagem

Alguns critérios p/ avaliação de lings.

- O custo da linguagem é um critério. Esse custo pode depender de vários fatores, como:
  - o custo para treinar um desenvolvedor naquela linguagem
  - o custo para escrever programas naquela linguagem
  - o custo para compilar e executar programas naquela linguagem
  - o custo de manutenção de programas naquela linguagem
  - dentre outros

Critérios de escolha de linguagem

- Quanto à implementação, observar se há disponibilidade quanto à plataforma e questões de velocidade de execução do programa objeto.

Critérios de escolha de linguagem

- Quanto à **implementação**, observar se há disponibilidade quanto à plataforma e questões de velocidade de execução do programa objeto.

- Quanto à **competência**, considerar a experiência dos programadores e das pessoas do grupo em geral.

Critérios de escolha de linguagem

- Quanto à **implementação**, observar se há disponibilidade quanto à plataforma e questões de velocidade de execução do programa objeto.

- Quanto à **competência**, considerar a experiência dos programadores e das pessoas do grupo em geral.

- Quanto à **portabilidade**, verificar a necessidade de se executar o programa em várias máquinas diferentes.

Critérios de escolha de linguagem

- Quanto à sintaxe, observar qual é a aplicação, já que algumas ficam melhor em certas sintaxes.

Critérios de escolha de linguagem

- Quanto à **sintaxe**, observar qual é a aplicação, já que algumas ficam melhor em certas sintaxes.

- Quanto à **semântica**, considerar aspectos como concorrência ou recursividade, já que algumas linguagens dão mais ou menos suporte à elas.

Critérios de escolha de linguagem

- Quanto à **sintaxe**, observar qual é a aplicação, já que algumas ficam melhor em certas sintaxes.

- Quanto à **semântica**, considerar aspectos como concorrência ou recursividade, já que algumas linguagens dão mais ou menos suporte à elas.

- Quanto ao **ambiente de programação**, observar a existência de ferramentas ou IDEs para acelerar o desenvolvimento e facilitá-lo.

Exemplos do que vimos hoje

- O problema é “ler um inteiro do teclado e atribuí-lo a uma variável inteira”. Como fazer isso em C, C++, Python ou Java?

```
C: int num; scanf(“%d”, &num);
C++: int num; cin >> num;
Python: num = int(input())
Java: int num;
      Scanner scan = new Scanner(System.in);
      num = scan.nextInt();
```

Compilação

- Na **compilação**, um programa escrito em uma linguagem é traduzido em uma linguagem de máquina, executado diretamente pelo computador.

Compilação

- Na **compilação**, um programa escrito em uma linguagem é traduzido em uma linguagem de máquina, executado diretamente pelo computador.

- Pode envolver a conversão em um **código intermediário**.

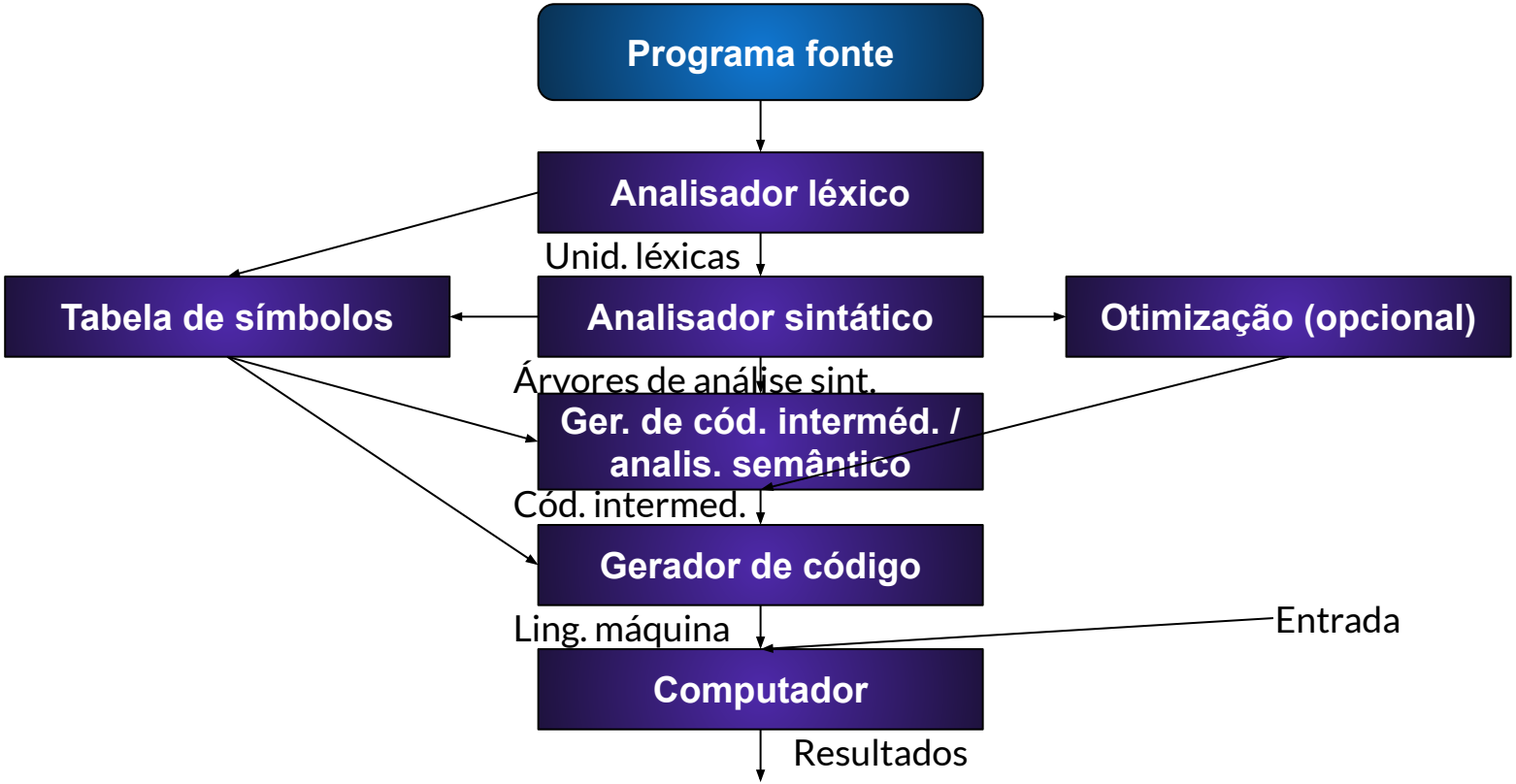
Compilação

- Na **compilação**, um programa escrito em uma linguagem é traduzido em uma linguagem de máquina, executado diretamente pelo computador.

- Pode envolver a conversão em um **código intermediário**.

- Normalmente, as linguagens compiladas permitem uma rápida execução do programa, após a tradução ter sido terminada.

Compilação


Programa fonte
Analisador léxico
Unid. léxicas
Tabela de símbolos
Analisador sintático
Otimização (opcional)
Árvores de análise sint.
Ger. de cód. interméd. / analis. semântico
Cód. intermed.
Gerador de código
Ling. máquina
Entrada
Computador
Resultados

Interpretação

- Na interpretação, um programa é traduzido por um interpretador, sem haver uma tradução.

Interpretação

- Na **interpretação**, um programa é traduzido por um interpretador, sem haver uma tradução.

- O interpretador age como uma simulação em software, tratando de sentenças em alto nível ao invés de instruções de máquina.

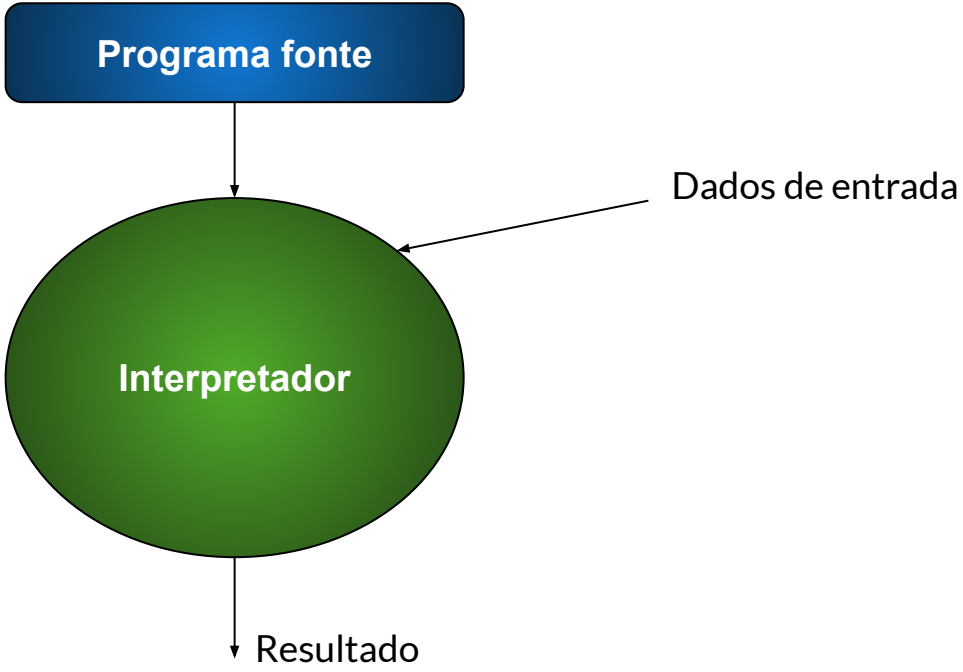
Interpretação

- Na **interpretação**, um programa é traduzido por um interpretador, sem haver uma tradução.

- O interpretador age como uma simulação em software, tratando de sentenças em alto nível ao invés de instruções de máquina.

- Pode facilitar algumas operações de depuração, mas em geral possui execução mais lenta.

Interpretação



Implementações híbridas

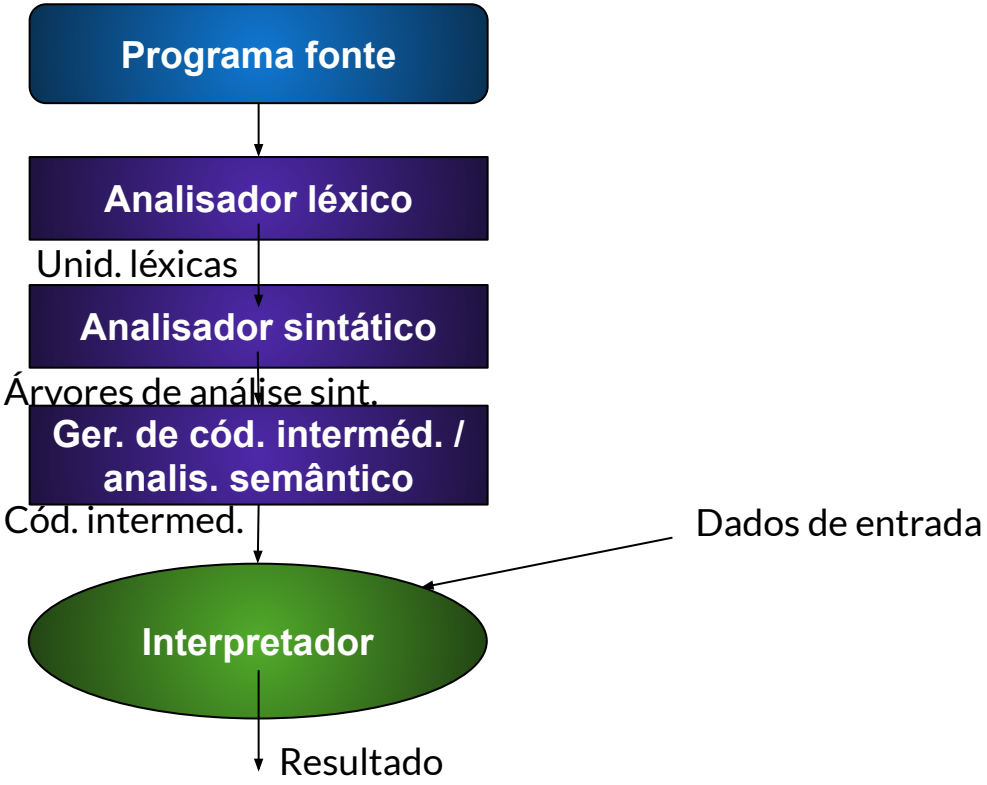
- É possível **combinar compilação e interpretação**. Nele, há a tradução da linguagem de alto nível para uma linguagem intermediária. O código dessa linguagem intermediária é, então, interpretado.

Implementações híbridas

- É possível **combinar compilação e interpretação**. Nele, há a tradução da linguagem de alto nível para uma linguagem intermediária. O código dessa linguagem intermediária é, então, interpretado.

- Desse modo, se consegue os benefícios das duas abordagens, como **execução mais rápida**, já que sentenças na linguagem fonte são decodificadas apenas uma vez, bem como **facilidades de depuração**.

Implementações híbridas



Implementações híbridas (Java)

- O código escrito em Java é traduzido em uma linguagem intermediária (*bytecode*). Qualquer máquina que tenha um **interpretador de bytecodes** e um **sistema de tempo de execução** roda esse código.

Implementações híbridas (Java)

- O código escrito em Java é traduzido em uma linguagem intermediária (*bytecode*). Qualquer máquina que tenha um **interpretador de bytecodes** e um **sistema de tempo de execução** roda esse código.

- No caso de Java, esses dois compõem a chamada **JVM** (*Java Virtual Machine*). As máquinas que possuam uma JVM podem executar código Java, o que colabora com a **portabilidade** da linguagem.

Java e compilação JIT (*just-in-time*)

- É possível uma abordagem mais rápida ainda:
  - nela, o programa é convertido em uma linguagem intermediária;

Java e compilação JIT (*just-in-time*)

- É possível uma abordagem mais rápida ainda:
  - nela, o programa é convertido em uma linguagem intermediária;
  - durante a execução, compila métodos da linguagem intermediária em linguagem de máquina quando são chamados;

Java e compilação JIT (*just-in-time*)

- É possível uma abordagem mais rápida ainda:
  - nela, o programa é convertido em uma linguagem intermediária;
  - durante a execução, compila métodos da linguagem intermediária em linguagem de máquina quando são chamados;
  - para chamadas futuras, a versão em linguagem de máquina do código é mantida.

Compiladas vs interpretadas

- Nas compiladas, o código é todo traduzido antes da execução. Nas interpretadas, isso ocorre linha p/ linha.

Link: https://www.educative.io/blog/compiled-vs-interpreted-language

Compiladas vs interpretadas

- Nas compiladas, o código é todo traduzido antes da execução. Nas interpretadas, isso ocorre linha p/ linha.

- O código binário gerado pelas compiladas depende da máquina. Nas interpretadas, isso não acontece.

Link: https://www.educative.io/blog/compiled-vs-interpreted-language

Compiladas vs interpretadas

- Nas compiladas, o código é todo traduzido antes da execução. Nas interpretadas, isso ocorre linha p/ linha.

- O código binário gerado pelas compiladas depende da máquina. Nas interpretadas, isso não acontece.

- As interpretadas facilitam algumas implementações (que veremos mais adiante), como tipagem dinâmica.

Link: https://www.educative.io/blog/compiled-vs-interpreted-language

Referências (válidas p/ demais slides)

O material da disciplina teve como bases, além do livro "Conceitos de Linguagens de Programação" do Robert Sebesta, os materiais de aula dos professores Janderson Aguiar (UEPB) e Franklin Ramalho (UFCG).

Referências

SEBESTA, Robert M. **Conceitos de Linguagens de Programação**. Bookman, 2018.